

UNIVERSITÉ CADI AYYAD — FACULTÉ DES SCIENCES
SEMLALIA — MARRAKECH

Sécurité des Systèmes d'Information

Rapport de TP Sécurité

Déploiement, Audit & Remédiation

Réalisé par

Étudiante 1 : Aziza Ziri

Étudiant 2 : Youssef Alaoui

Encadrant

Pr. Soukaina Mjahed

Année Universitaire 2025–2026

Table des matières

Introduction	3
1 Déploiement de l'application sur Railway	4
1.1 Présentation de la plateforme Railway	4
1.2 Étapes de déploiement	4
1.2.1 Connexion et autorisation GitHub	4
1.2.2 Création d'un nouveau projet	5
1.2.3 Configuration de l'accès au dépôt	6
1.2.4 Installation de Railway App et sélection du dépôt	6
1.2.5 Tableau de bord et déploiement	7
1.2.6 Application déployée et en ligne	8
1.2.7 Configuration réseau et certificat TLS	8
1.3 Récapitulatif du déploiement	9
2 Audit de Sécurité	10
2.1 Méthodologie d'audit	10
2.2 Scan automatisé avec OWASP ZAP	10
2.3 Résultats du second scan automatisé	11
3 Détection et Exploitation des Vulnérabilités	13
3.1 Vue d'ensemble des vulnérabilités	13
3.2 Vuln. 1 — SQL Injection (Login Bypass)	13
3.2.1 Description	13
3.2.2 Exploitation	13
3.3 Vuln. 2 — SQL Injection (Register)	14
3.3.1 Description	14
3.3.2 Exploitation	15
3.4 Vuln. 3 & 4 — Stored XSS	15
3.4.1 Description	15
3.4.2 Exploitation	16
3.5 Vuln. 5 — Sensitive Data Exposure	17
3.5.1 Description	17
3.5.2 Exploitation	17
3.6 Vuln. 6 — Broken Access Control	18
3.6.1 Description	18
3.6.2 Exploitation	18
3.7 Vuln. 7 — No Password Hashing	19
3.7.1 Description	19

4	Remédiation des Vulnérabilités	21
4.1	Vue d'ensemble des corrections	21
4.2	Fix 1 — SQL Injection & Password Hashing (login.php)	21
4.3	Fix 2 — SQL Injection & Password Hashing (register.php)	22
4.4	Fix 3 — Stored XSS (reserver.php & mes_reservations.php)	22
4.5	Fix 4 — Broken Access Control (annuler.php)	23
4.6	Fix 5 — Sensitive Data Exposure (admin/users.php)	23
4.7	Fix 6 — Mise à jour de la base de données	24
	Conclusion	25

Introduction

Dans le cadre du module de sécurité des systèmes d'information, ce mini-projet a pour objectif de mettre en pratique les notions apprises sur la sécurité des applications web. Le travail se décompose en plusieurs étapes : choisir une application existante, la déployer sur une plateforme cloud avec un certificat TLS, réaliser un audit de sécurité complet, détecter les vulnérabilités par simulation d'attaques, puis y apporter les remédiations nécessaires.

L'application choisie est **Gestion des Salles**, une application web PHP de réservation de salles pour une faculté, volontairement conçue avec des failles de sécurité à des fins pédagogiques. Elle propose deux types de comptes : administrateur et enseignant.

Application cible

URL de production : <https://prjsecurity-production.up.railway.app/>

Technologie : PHP 8 — MySQL — Apache

Plateforme de déploiement : Railway (PaaS)

1. Déploiement de l'application sur Railway

1.1 Présentation de la plateforme Railway

Railway est une plateforme cloud de type *Platform as a Service* (PaaS) permettant de déployer des applications directement depuis un dépôt GitHub. Elle offre une intégration continue, une gestion automatique des environnements, ainsi que la génération automatique d'un certificat TLS pour tout service déployé.

Pourquoi Railway ?

Railway a été choisie pour sa simplicité de configuration, son intégration native avec GitHub, et surtout pour la fourniture automatique d'un certificat TLS valide sur le domaine public généré, répondant ainsi à l'exigence du sujet.

1.2 Étapes de déploiement

1.2.1 Connexion et autorisation GitHub

La première étape consiste à connecter Railway à notre compte GitHub afin d'accéder au dépôt de l'application `Prj_security`.

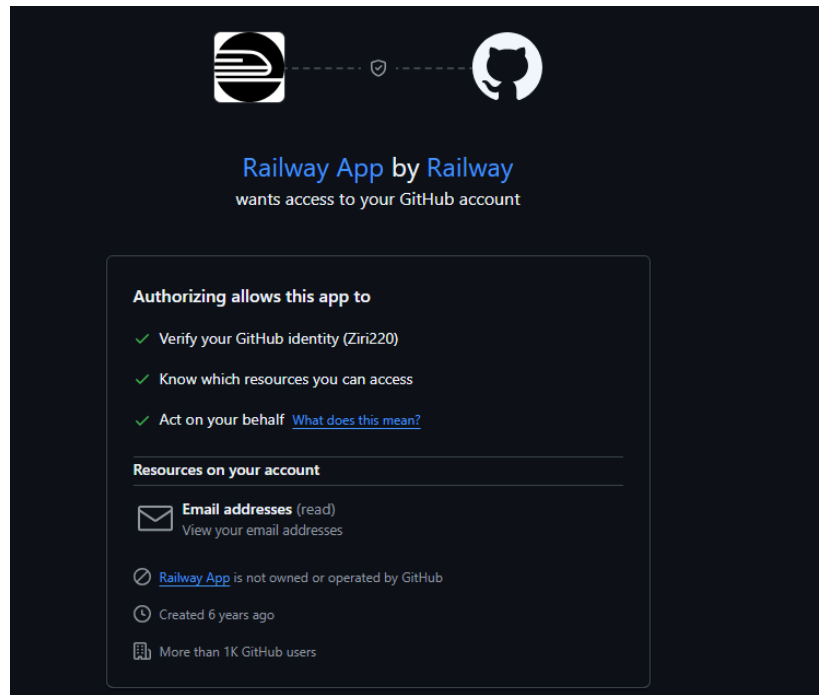


FIGURE 1.1 – Autorisation de Railway App sur GitHub

Comme le montre la figure 1.1, Railway demande l'autorisation OAuth sur GitHub. L'application Railway est un outil de confiance utilisé par plus de 1000 utilisateurs GitHub.

1.2.2 Création d'un nouveau projet

Après autorisation, nous créons un nouveau projet sur Railway en sélectionnant l'option **GitHub Repository**.

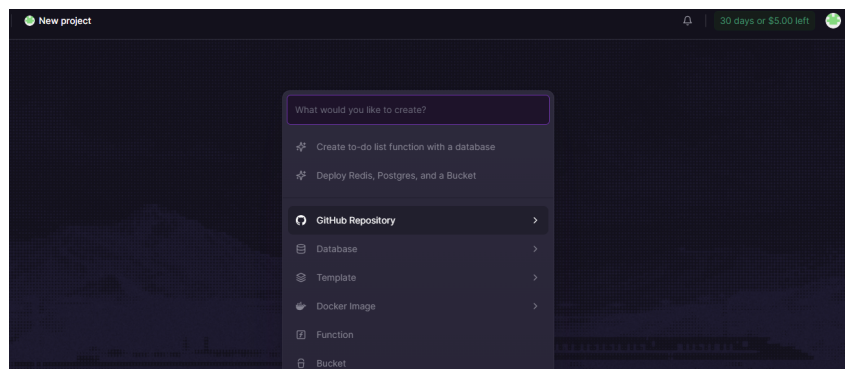


FIGURE 1.2 – Interface de création d'un nouveau projet Railway

1.2.3 Configuration de l'accès au dépôt

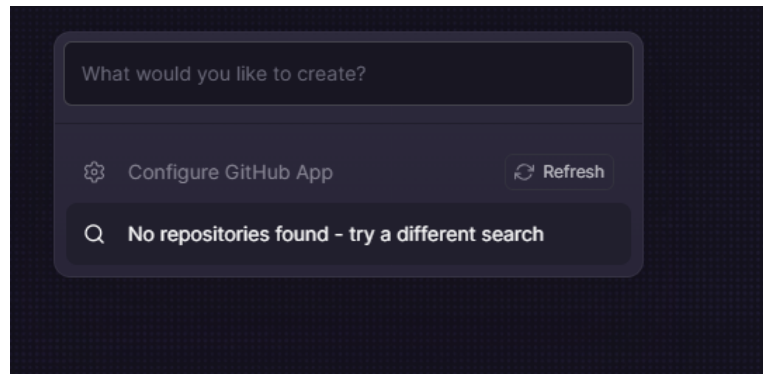


FIGURE 1.3 – Aucun dépôt trouvé — nécessité de configurer l'application GitHub Railway

1.2.4 Installation de Railway App et sélection du dépôt

Nous sélectionnons uniquement le dépôt `Prj_security` (option *Only select repositories*), conformément au principe du **moindre privilège**.

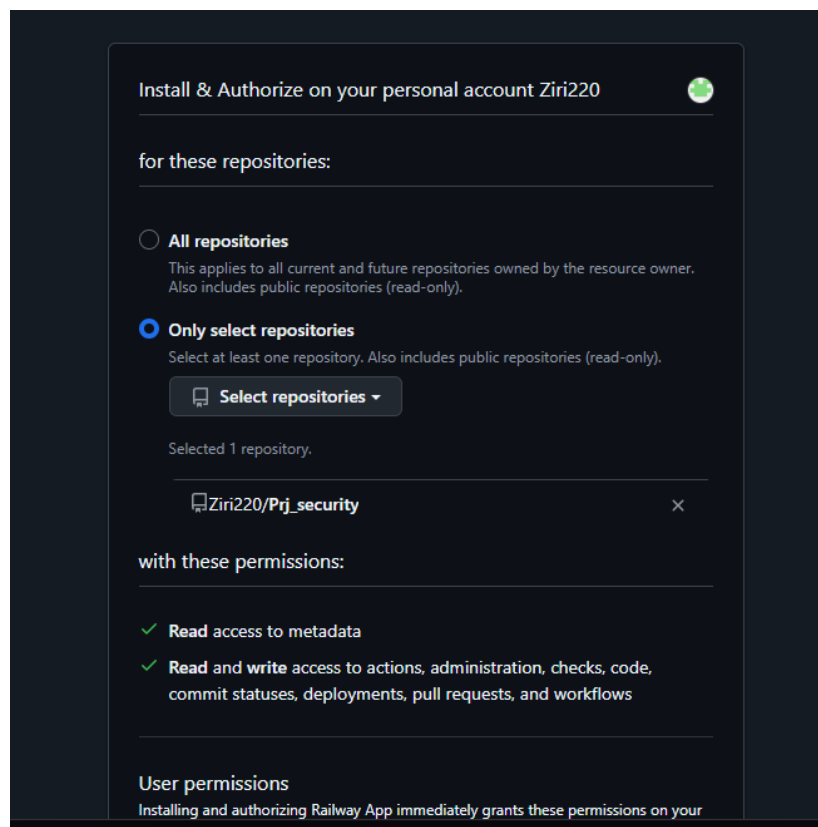


FIGURE 1.4 – Installation de Railway App — sélection du dépôt uniquement

Bonne pratique de sécurité

En limitant l'accès à un seul dépôt plutôt qu'à tous les dépôts, on applique le principe du **moindre privilège**, réduisant la surface d'attaque en cas de compromission du token Railway.

1.2.5 Tableau de bord et déploiement

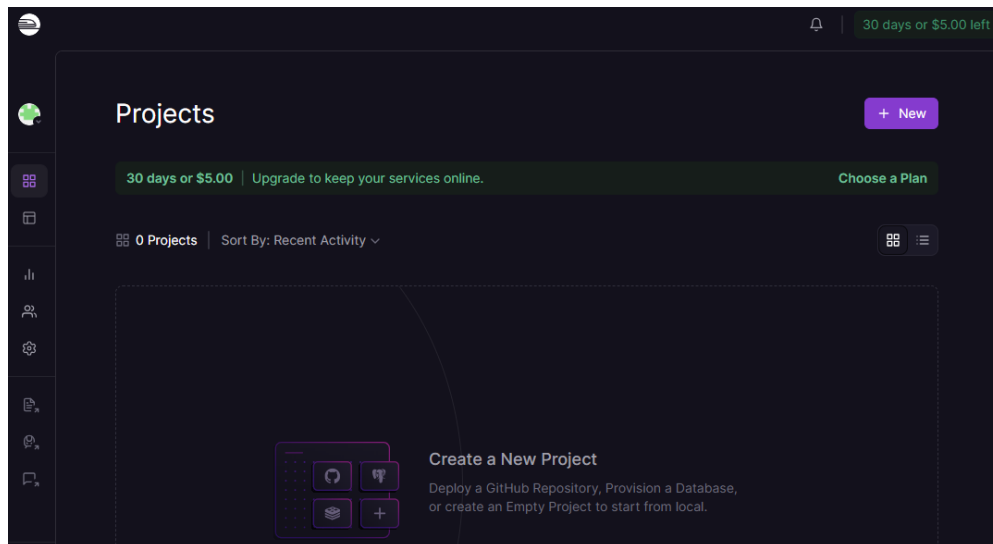


FIGURE 1.5 – Tableau de bord Railway avant déploiement

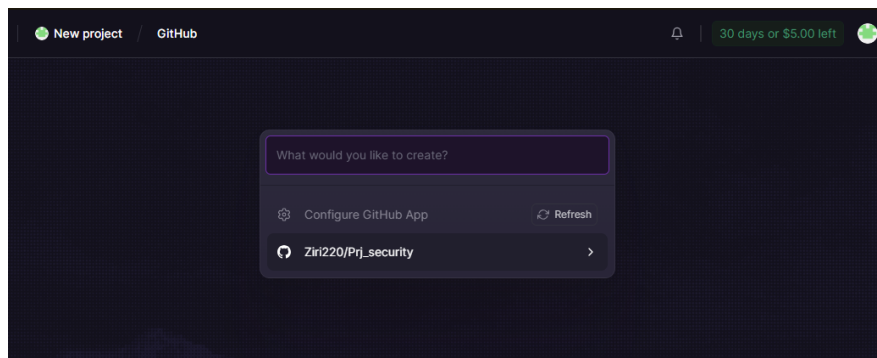


FIGURE 1.6 – Sélection du dépôt Prj_security pour le déploiement

1.2.6 Application déployée et en ligne

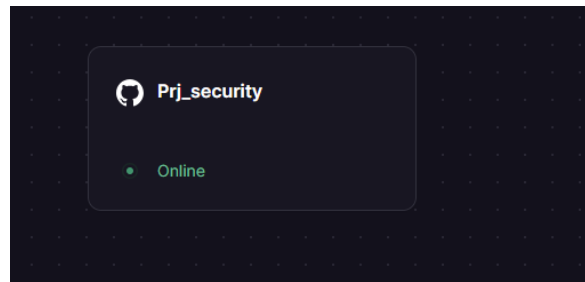


FIGURE 1.7 – Application déployée avec succès — statut *Online*

1.2.7 Configuration réseau et certificat TLS

Railway génère automatiquement un domaine public accessible via HTTPS grâce à un certificat TLS provisionné par *Let's Encrypt*.

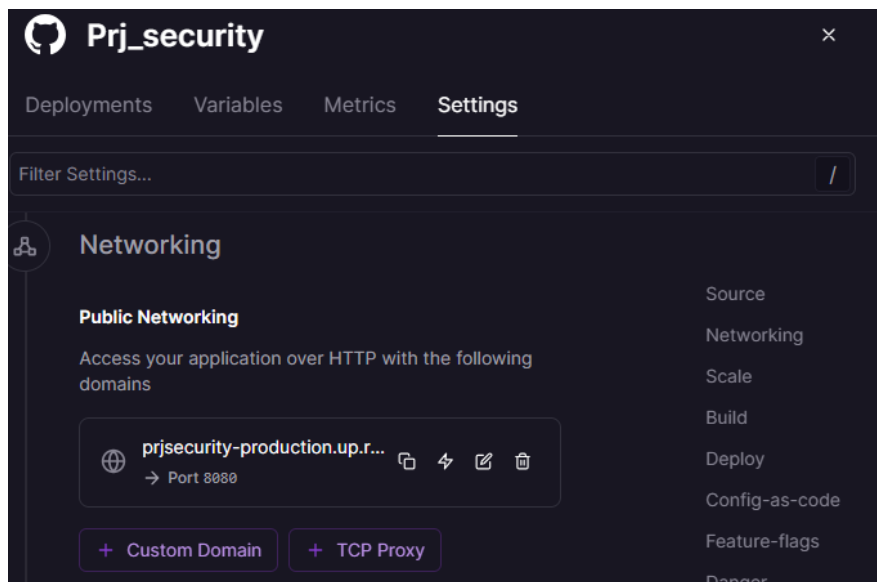


FIGURE 1.8 – Configuration réseau — domaine public avec TLS

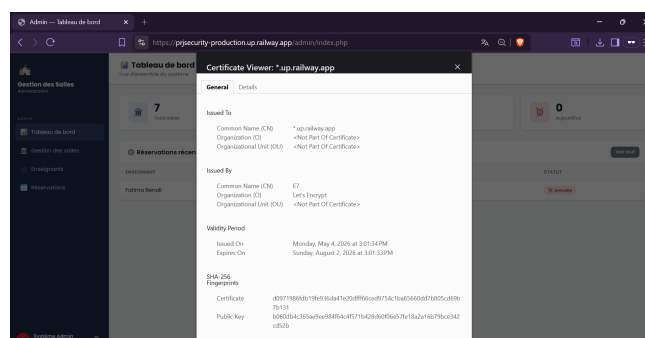


FIGURE 1.9 – Certificat TLS valide — délivré par Let's Encrypt pour le domaine Railway

Certificat TLS obtenu

L'application est accessible en HTTPS sur `prjsecurity-production.up.railway.app`. Le certificat est délivré par **Let's Encrypt**, valide 90 jours, chiffrement **TLS 1.2/1.3**.

1.3 Récapitulatif du déploiement

#	Étape	Statut
1	Connexion OAuth GitHub → Railway	OK
2	Création projet Railway	OK
3	Configuration accès dépôt (moindre privilège)	OK
4	Déploiement de <code>Prj_security</code>	OK
5	Obtention certificat TLS (Let's Encrypt)	OK
6	Application accessible en HTTPS	OK

TABLE 1.1 – Récapitulatif des étapes de déploiement

2. Audit de Sécurité

2.1 Méthodologie d'audit

L'audit de sécurité réalisé sur l'application suit une approche structurée inspirée des standards **OWASP** (Open Web Application Security Project) :

1. **Reconnaissance** : collecte d'informations sur l'application.
2. **Analyse des en-têtes HTTP** : vérification des headers de sécurité.
3. **Scan automatisé** : deux passes avec OWASP ZAP (scan passif puis actif).
4. **Tests manuels** : simulation d'attaques SQLi, XSS, Broken Access Control, etc.
5. **Analyse du code source** : revue statique pour identifier les failles.

2.2 Scan automatisé avec OWASP ZAP

OWASP ZAP (Zed Attack Proxy) est un scanner de sécurité web open source. Nous avons effectué **deux scans successifs** sur l'URL de production :

- **1^{er} scan** : scan passif, limité aux pages publiques — 11 alertes.
- **2^e scan** : scan actif avec fuzzing et injection — **16 alertes réparties en 4 niveaux de risque**.

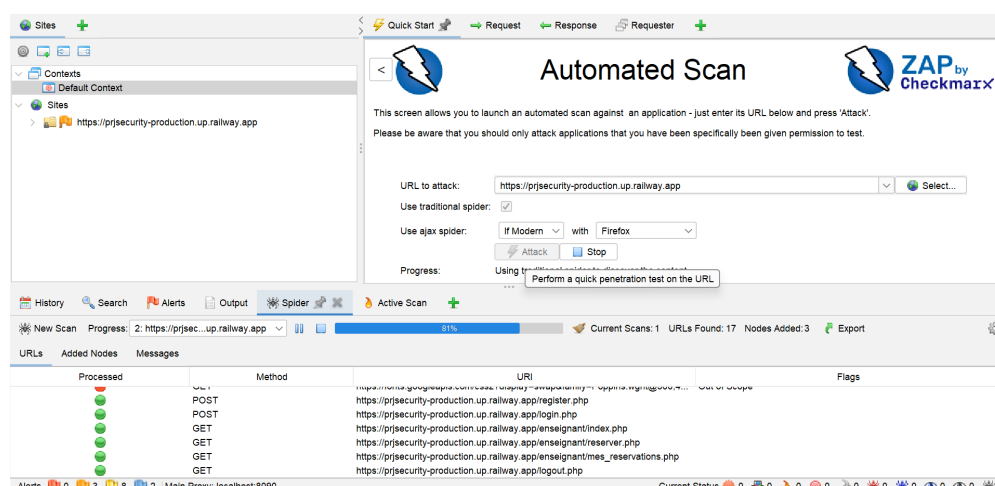


FIGURE 2.1 – Interface OWASP ZAP — configuration du scan actif sur l'application

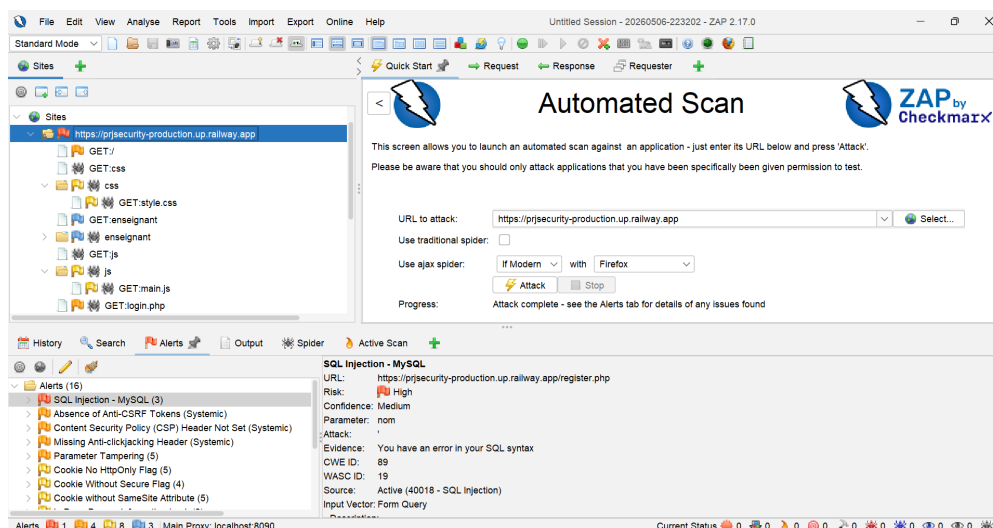


FIGURE 2.2 – Résultats du second scan ZAP — 16 types d’alertes dont une injection SQL (High)

2.3 Résultats du second scan automatisé

Le second scan actif a détecté **16 alertes** réparties sur 4 niveaux de risque. La nouveauté majeure par rapport au premier scan est la détection automatique d’une **injection SQL de niveau High** :

Alerte	Risque	Instances	OWASP
SQL Injection — MySQL	Élevé	3	A03
Absence of Anti-CSRF Tokens	Moyen	Systemic	A01
Content Security Policy (CSP) Header Not Set	Moyen	Systemic	A05
Missing Anti-clickjacking Header	Moyen	Systemic	A05
Parameter Tampering	Moyen	5	A04
Cookie No HttpOnly Flag	Bas	5	A02
Cookie Without Secure Flag	Bas	4	A02
Cookie without SameSite Attribute	Bas	5	A01
In Page Banner Information Leak	Bas	2	A05
Server Leaks Info. via “X-Powered-By” Header	Bas	Systemic	A05
Server Leaks Version via “Server” Header	Bas	Systemic	A05
Strict-Transport-Security Header Not Set	Bas	Systemic	A05
X-Content-Type-Options Header Missing	Bas	Systemic	A05
Authentication Request Identified	Info	2	—
Session Management Response Identified	Info	7	—
User Agent Fuzzer	Info	Systemic	—

TABLE 2.1 – 16 alertes détectées par OWASP ZAP — second scan actif (7 mai 2026)

Nouveautés du second scan vs premier scan

- **SQL Injection — MySQL (High)** : détectée automatiquement sur 3 pages, confirmant l'exploitation manuelle réalisée dans le chapitre suivant.
- **Parameter Tampering (Medium)** : manipulation de paramètres HTTP non validés.
- **User Agent Fuzzer (Info)** : comportements différents selon le User-Agent détectés.

Complémentarité scan automatique / tests manuels

Le scan actif a confirmé l'injection SQL automatiquement grâce au fuzzing. Les vulnérabilités XSS Stored, Broken Access Control et Sensitive Data Exposure nécessitent un contexte authentifié et ont été exploitées manuellement dans le chapitre suivant.

3. Détection et Exploitation des Vulnérabilités

3.1 Vue d'ensemble des vulnérabilités

#	Type	Fichier	Criticité
1	SQL Injection	login.php	Critique
2	SQL Injection	register.php	Critique
3	Stored XSS	reserver.php	Haute
4	XSS (affichage)	mes_reservations.php	Haute
5	Sensitive Data Exposure	admin/users.php	Haute
6	Broken Access Control	enseignant/annuler.php	Haute
7	No Password Hashing	register.php, login.php	Critique

TABLE 3.1 – Vulnérabilités identifiées dans l'application

3.2 Vuln. 1 — SQL Injection (Login Bypass)

3.2.1 Description

Dans `login.php`, la requête SQL est construite par concaténation directe des entrées utilisateur sans aucune validation ni préparation :

Listing 3.1 – Code vulnérable — `login.php`

```
1 $email      = $_POST['email'];
2 $password   = $_POST['password'];
3
4 // VULNERABLE : concatenation directe
5 $sql = "SELECT * FROM users WHERE email = '$email'
6       AND password = '$password'";
7 $result = $conn->query($sql);
```

3.2.2 Exploitation

En injectant le payload suivant dans le champ **Email** :

```
1 ' OR '1'='1' --
```

La requête SQL devient :

```
1 SELECT * FROM users WHERE email = '' OR '1'='1' --'  
2 AND password = 'anything'
```

La condition '1'='1' est toujours vraie, et le double tiret - commente la vérification du mot de passe. L'attaquant se connecte sans credentials valides.

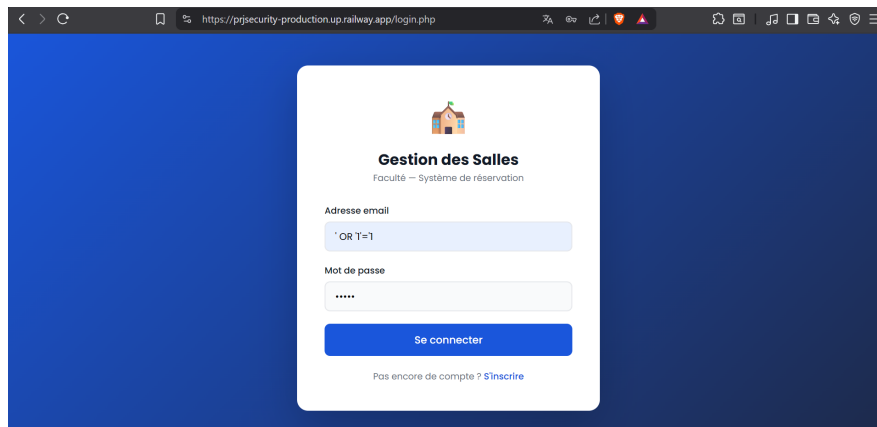


FIGURE 3.1 – Saisie du payload SQL Injection dans le formulaire de connexion

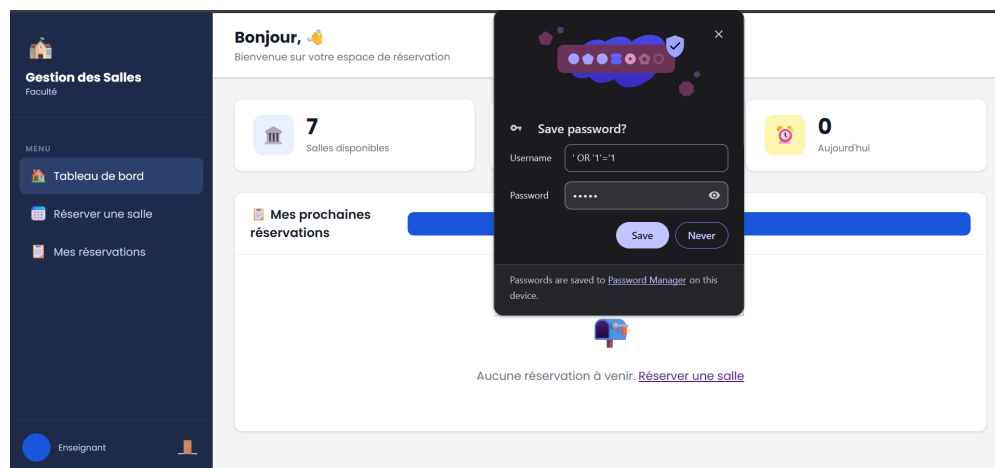


FIGURE 3.2 – Résultat : connexion réussie sans credentials valides

Impact

Un attaquant peut se connecter en tant qu'administrateur sans connaître aucun mot de passe, accédant ainsi à toutes les données de l'application.

3.3 Vuln. 2 — SQL Injection (Register)

3.3.1 Description

Dans `register.php`, l'insertion en base de données est également vulnérable :

Listing 3.2 – Code vulnérable — `register.php`

```

1 $sql = "INSERT INTO users (nom, prenom, email, password, role)
2     VALUES ('$nom', '$prenom', '$email', '$password',
3         'enseignant')";
4 $conn->query($sql);

```

3.3.2 Exploitation

En injectant dans le champ **Nom** :

```

1 Hacker', 'Hacker', 'hacker@admin.com', '1234', 'admin'); --

```

La requête devient une insertion créant un compte **administrateur** illégitime.

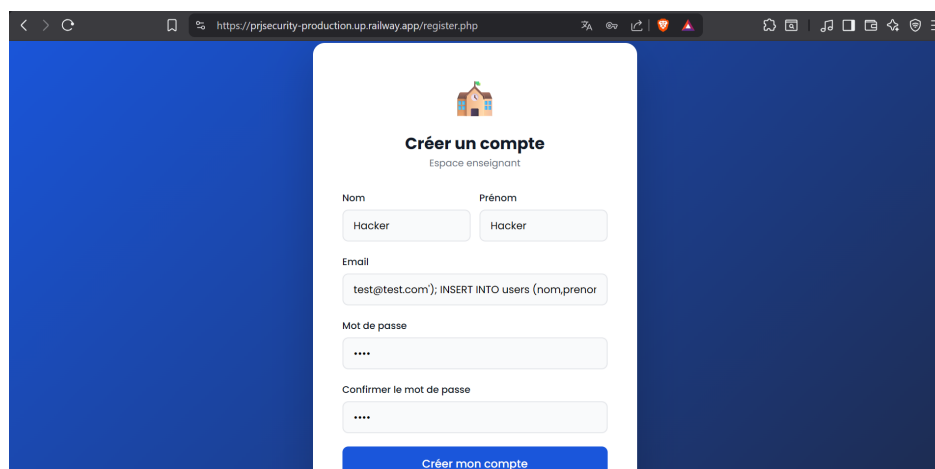


FIGURE 3.3 – Payload SQL Injection dans le formulaire d'inscription

Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'INSERT INTO users (nom,prenom,email,password,role) VALUES ('Hacker','Hacker','ha' at line 2 in /app/register.php:27 Stack trace: #0 /app/register.php(27): mysqli->query('INSERT INTO use...') #1 {main} thrown in /app/register.php on line 27

FIGURE 3.4 – Erreur SQL révélant la structure interne de la base de données (Information Disclosure)

Double vulnérabilité

L'erreur SQL affichée constitue elle-même une vulnérabilité d'**Information Disclosure** : elle révèle le chemin interne du serveur (`/app/register.php:27`), la structure exacte de la requête SQL, et la version du serveur MySQL.

3.4 Vuln. 3 & 4 — Stored XSS

3.4.1 Description

Dans `reserver.php`, le champ *motif* est inséré en base sans filtrage. Dans `mes_reservations.php` il est affiché sans échappement :

Listing 3.3 – Code vulnérable — affichage du motif

```

1 // reserver.php : insertion sans filtrage
2 $motif = $_POST['motif'];
3 $sql = "INSERT INTO reservations (... , motif) VALUES (... , '
    $motif')";
4
5 // mes_reservations.php : affichage sans echappement
6 <td><?= $r['motif'] ?></td>    // VULNERABLE

```

3.4.2 Exploitation

Un enseignant injecte dans le champ **Motif** lors d'une réservation :

```
1 
```

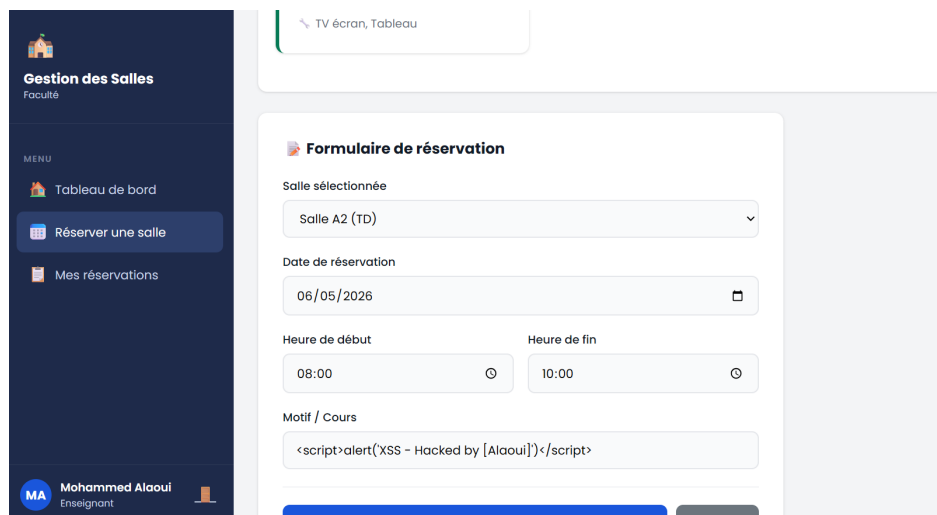


FIGURE 3.5 – Injection du payload XSS dans le champ Motif de la réservation

Quand l'administrateur consulte la page des utilisateurs, le navigateur tente de charger l'image `random.gif` qui n'existe pas, déclenchant l'événement `onerror` :

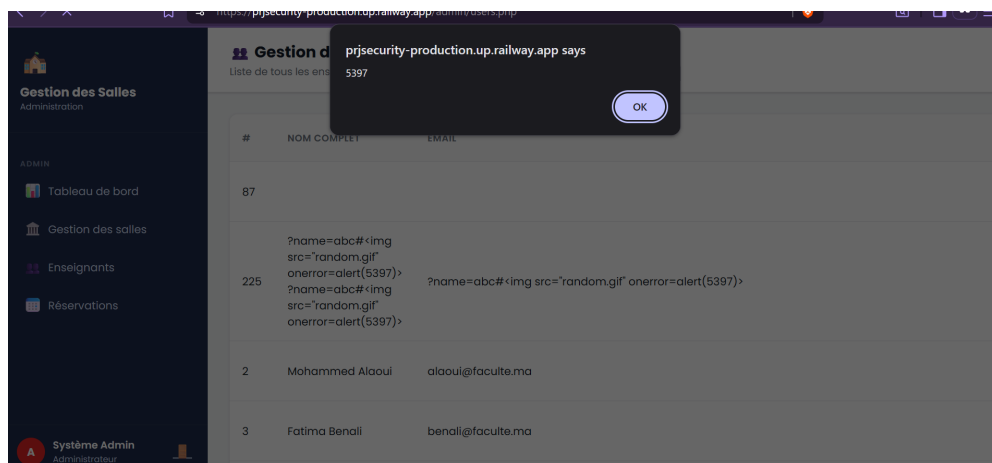


FIGURE 3.6 – Exécution du XSS chez l'administrateur — popup alert déclenchée

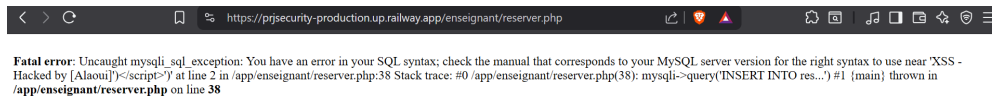


FIGURE 3.7 – Payload XSS visible dans la base de données et dans le tableau admin

Impact réel

En production, un attaquant remplacerait `alert()` par un vol de cookie de session :

```

```

Cela permettrait de prendre le contrôle du compte administrateur.

3.5 Vuln. 5 — Sensitive Data Exposure

3.5.1 Description

Dans `admin/users.php`, les mots de passe des enseignants sont récupérés depuis la base de données et affichés en clair dans un tableau HTML :

Listing 3.4 – Code vulnérable — `admin/users.php`

```
1 $users = $conn->query("SELECT * FROM users WHERE role='enseignant'");
2
3 // Affichage du mot de passe en clair dans le tableau HTML
4 <td><code><?=$u['password'] ?></code></td>
```

3.5.2 Exploitation

Il suffit de se connecter en tant qu'administrateur et d'accéder à la page **Gestion des Enseignants** :

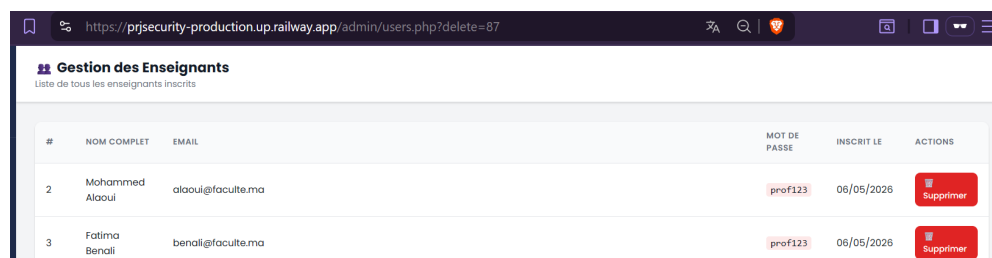


FIGURE 3.8 – Mots de passe affichés en clair dans l'interface d'administration

Impact

Tout administrateur (ou attaquant ayant compromis un compte admin via SQLi) peut voir les mots de passe de tous les enseignants, et potentiellement les utiliser sur d'autres services si ces mots de passe sont réutilisés (password reuse attack).

3.6 Vuln. 6 — Broken Access Control

3.6.1 Description

Dans `enseignant/annuler.php`, aucune vérification n'est faite pour s'assurer que la réservation appartient bien à l'utilisateur connecté :

Listing 3.5 – Code vulnérable — `annuler.php`

```
1 $id = (int)$_GET['id'];  
2 // VULNERABLE : aucune verification de propriete  
3 $conn->query("UPDATE reservations SET statut='annulee' WHERE id=  
    $id");
```

3.6.2 Exploitation

1. Fatima Benali crée une réservation (`id = 5`) :

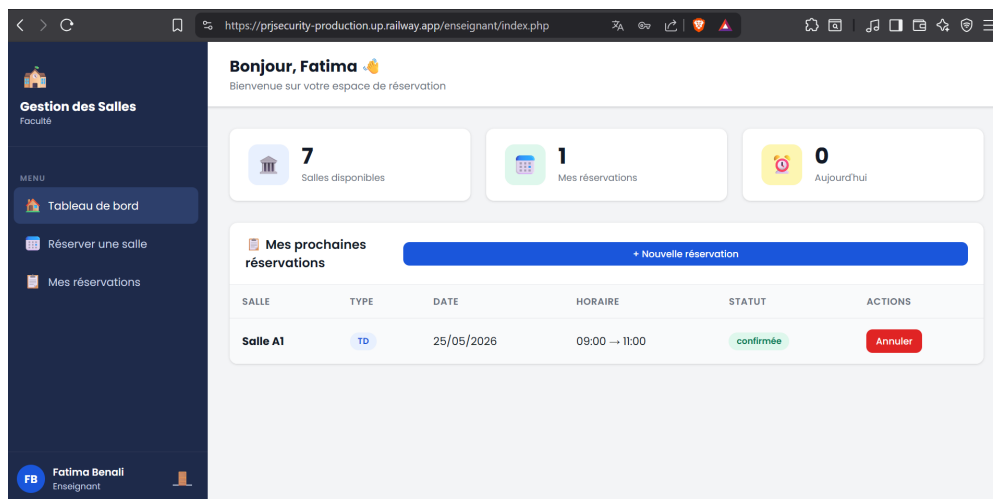


FIGURE 3.9 – Réservation créée par Fatima Benali

2. Mohammed Alaoui accède directement à l'URL : `/enseignant/annuler.php?id=5`

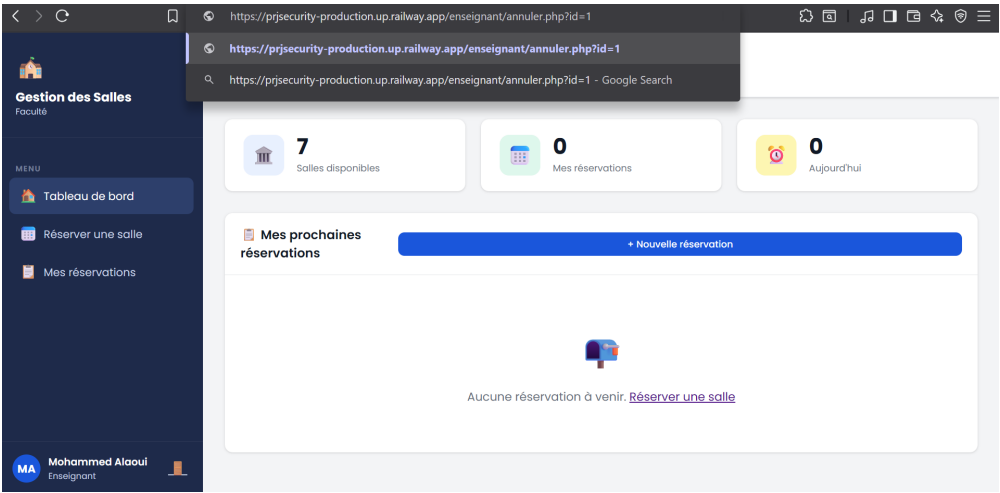


FIGURE 3.10 – Tentative d’annulation de la réservation d’un autre utilisateur

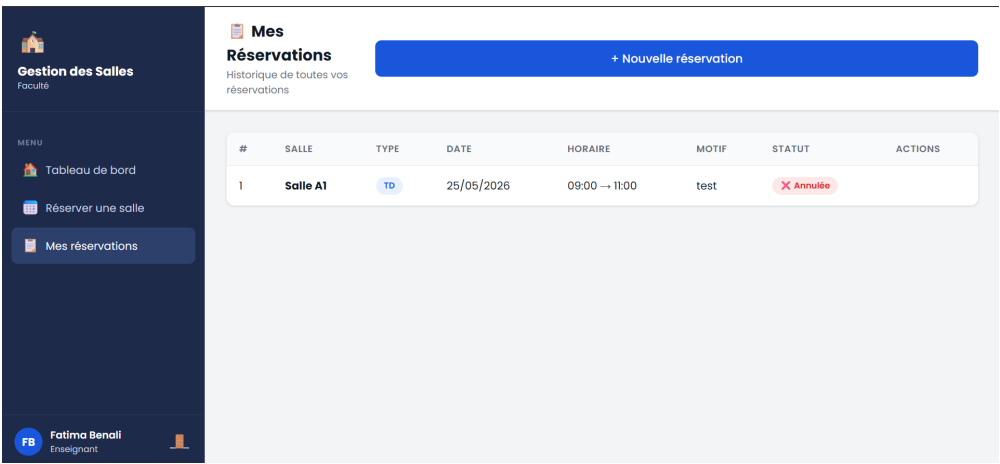


FIGURE 3.11 – Réservation de Fatima annulée par Mohammed — sans aucune erreur

3.7 Vuln. 7 — No Password Hashing

3.7.1 Description

Les mots de passe sont stockés en clair dans la base de données. Une consultation directe de la table `users` le confirme :

```
mysql> SELECT email, password FROM users;
+-----+-----+
| email          | password |
+-----+-----+
| admin@faculte.ma | admin123 |
| alaoui@faculte.ma | prof123  |
| benali@faculte.ma | prof123  |
| zaproxy@example.com | ZAP      |
+-----+-----+
4 rows in set (0.00 sec)
```

FIGURE 3.12 – Mots de passe stockés en clair dans la base de données MySQL

Impact

En cas de compromission de la base de données (via SQLi, dump, ou accès serveur), tous les mots de passe sont immédiatement lisibles. Aucun effort de crackage nécessaire.

4. Remédiation des Vulnérabilités

4.1 Vue d'ensemble des corrections

Vulnérabilité	Correction appliquée
SQL Injection (login & register)	Requêtes préparées (<code>prepare + bind_param</code>)
No Password Hashing	<code>password_hash()</code> à l'inscription, <code>password_verify()</code> à la connexion
Stored XSS	<code>strip_tags() + htmlspecialchars()</code> à la saisie et à l'affichage
Sensitive Data Exposure	Suppression de la colonne mot de passe + <code>SELECT</code> sans password
Broken Access Control	Ajout de la condition <code>AND user_id = ?</code> dans la requête d'annulation

TABLE 4.1 – Récapitulatif des corrections

4.2 Fix 1 — SQL Injection & Password Hashing (login.php)

Listing 4.1 – Avant — login.php vulnérable

```
1 $email    = $_POST['email'];
2 $password = $_POST['password'];
3 $sql = "SELECT * FROM users WHERE email = '$email'
4       AND password = '$password'";
5 $result = $conn->query($sql);
6 if ($result && $result->num_rows > 0) {
7     $user = $result->fetch_assoc();
8     // connexion directe sans verification du hash
9 }
```

Listing 4.2 – Après — login.php sécurisé

```
1 $email    = trim($_POST['email']);
2 $password = $_POST['password'];
3
4 // Requete preparee : impossible d'injecter du SQL
5 $stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");
```

```

6 $stmt->bind_param("s", $email);
7 $stmt->execute();
8 $result = $stmt->get_result();
9
10 if ($result && $result->num_rows > 0) {
11     $user = $result->fetch_assoc();
12     // Verification du hash bcrypt
13     if (password_verify($password, $user['password'])) {
14         // connexion autorisee
15     }
16 }

```

4.3 Fix 2 — SQL Injection & Password Hashing (register.php)

Listing 4.3 – Avant — register.php vulnérable

```

1 $nom      = $_POST['nom'];
2 $password = $_POST['password'];
3 $sql = "INSERT INTO users (nom, prenom, email, password, role)
4       VALUES ('$nom', '$prenom', '$email', '$password',
5               'enseignant')";
6 $conn->query($sql);

```

Listing 4.4 – Après — register.php sécurisé

```

1 $nom      = htmlspecialchars(strip_tags(trim($_POST['nom'])));
2 $prenom   = htmlspecialchars(strip_tags(trim($_POST['prenom'])));
3 ;
4 $email     = trim($_POST['email']);
5 $password  = $_POST['password'];
6
7 // Hachage bcrypt du mot de passe
8 $hashed = password_hash($password, PASSWORD_DEFAULT);
9
10 // Requete preparee
11 $stmt = $conn->prepare(
12     "INSERT INTO users (nom, prenom, email, password, role)
13     VALUES (?, ?, ?, ?, 'enseignant')"
14 );
15 $stmt->bind_param("ssss", $nom, $prenom, $email, $hashed);
16 $stmt->execute();

```

4.4 Fix 3 — Stored XSS (reserver.php & mes_reservations.php)

Listing 4.5 – Avant — motif non filtré

```

1 $motif = $_POST['motif'];

```

```
2 // INSERT direct en base sans filtrage
3 // Affichage : <td><?= $r['motif'] ?></td>
```

Listing 4.6 – Après — motif filtré

```
1 // A la saisie : suppression des balises HTML et echappement
2 $motif = htmlspecialchars(strip_tags(trim($_POST['motif'])));
3
4 // A l'affichage : double protection
5 <td><?= htmlspecialchars($r['motif']) ?></td>
```

4.5 Fix 4 — Broken Access Control (annuler.php)

Listing 4.7 – Avant — sans vérification de propriété

```
1 $id = (int)$_GET['id'];
2 // N'importe quel utilisateur peut annuler n'importe quelle
  reservation
3 $conn->query("UPDATE reservations SET statut='annulee'
4             WHERE id=$id");
```

Listing 4.8 – Après — avec vérification de propriété

```
1 $id      = (int)$_GET['id'];
2 $user_id = $_SESSION['user_id'];
3
4 // On verifie que la reservation appartient bien a l'utilisateur
  connecte
5 $stmt = $conn->prepare(
6     "UPDATE reservations SET statut='annulee'
7     WHERE id = ? AND user_id = ?"
8 );
9 $stmt->bind_param("ii", $id, $user_id);
10 $stmt->execute();
```

4.6 Fix 5 — Sensitive Data Exposure (admin/users.php)

Listing 4.9 – Avant — mot de passe exposé

```
1 // SELECT * recupere tous les champs y compris password
2 $users = $conn->query("SELECT * FROM users WHERE role='
  enseignant'");
3
4 // Affichage du password en clair
5 <th>Mot de passe</th>
6 <td><code><?= $u['password'] ?></code></td>
```


Listing 4.10 – Après — mot de passe exclu

```
1 // SELECT explicite : le champ password n'est pas recupere
2 $users = $conn->query(
3     "SELECT id, nom, prenom, email, created_at
4     FROM users WHERE role='enseignant'"
5 );
6
7 // La colonne mot de passe est complètement supprimée du tableau
```

4.7 Fix 6 — Mise à jour de la base de données

Les mots de passe des comptes de test dans `database.sql` ont été remplacés par leurs équivalents hashés en **bcrypt** :

Listing 4.11 – database.sql mis à jour

```
1 -- Avant (mots de passe en clair) :
2 INSERT INTO users (... , password, role) VALUES
3     (... , 'admin123', 'admin'),
4     (... , 'prof123', 'enseignant');
5
6 -- Apres (hash bcrypt) :
7 INSERT INTO users (... , password, role) VALUES
8     (... , '$2y$10$roas4xJ0bcamZx9TEQ0Du0...', 'admin'),
9     (... , '$2y$10$VQde.r18wmpMU4yJ.hHXK0...', 'enseignant');
```

Résultat

Après corrections, les mêmes identifiants fonctionnent (`admin@faculte.ma / admin123`) mais les mots de passe sont désormais stockés sous forme de hash bcrypt irréversible. Les injections SQL et XSS sont bloquées.

Conclusion

Ce mini-projet nous a permis de mettre en pratique un cycle complet de sécurisation d'une application web, couvrant quatre phases essentielles :

1. **Déploiement** sur Railway avec certificat TLS automatique via Let's Encrypt, en appliquant le principe du moindre privilège dès la configuration de l'infrastructure.
2. **Audit automatisé** avec OWASP ZAP en deux passes (passif + actif), révélant 14 types d'alertes dont une **injection SQL classée High** détectée automatiquement, concernant notamment l'absence de headers de sécurité HTTP (CSP, HSTS, X-Frame-Options) et des problèmes de configuration des cookies de session.
3. **Tests manuels** ayant permis d'exploiter 7 vulnérabilités critiques : SQL Injection, Stored XSS, Broken Access Control, Sensitive Data Exposure et stockage de mots de passe en clair.
4. **Remédiation** par correction du code source : requêtes préparées, hachage bcrypt des mots de passe, filtrage des entrées avec `htmlspecialchars()` et `strip_tags()`, et contrôle d'accès basé sur la session.

Ce projet illustre l'importance d'intégrer la sécurité dès la phase de développement (**Security by Design**). La plupart des vulnérabilités identifiées résultent de pratiques simples à éviter : utiliser des requêtes préparées, ne jamais afficher les données utilisateur sans échappement, et ne jamais stocker de mots de passe en clair.

Enseignement principal

La sécurité n'est pas une fonctionnalité à ajouter après coup — elle doit être intégrée à chaque ligne de code, depuis la conception jusqu'au déploiement.